Binary Face Mask Classification Using Deep Learning

Noran Aljodi, Rema Althaqfi, Enas Althyabi, Asail Al-Osaimi and Reema Alharbi
Department of Computer Science and Artificial Intelligence, College of Computing, Umm Al-Qura University, KSA

ABSTRACT

This study presents a deep learning framework for binary face mask classification using transfer learning with a pre-trained ResNet50 architecture, evaluated on the Face Mask 12K Images Dataset consisting of 11,792 facial images. The baseline model was established by freezing all feature extraction layers and replacing the final classification layer with a two-neuron softmax head, trained with a learning rate of 0.001. A systematic experimental study of 12 configurations was conducted to evaluate the independent effects of training sample size, number of epochs, and learning rate on model performance. The optimal baseline configuration achieved a test accuracy of 98.89%, a test loss of 0.0312, and an AUC of 0.9993.

Building upon the baseline, a series of improvement experiments were conducted across three categories. First, architectural regularization through Batch Normalization, Dropout ($p = 0.2$), and L2 weight decay was introduced to mitigate overfitting, yielding a stable accuracy of 94.35%. Second, image resolution and color space modifications were evaluated, revealing that high-resolution input (300×300) maintained near-baseline accuracy at 98.79%, low-resolution input (108×108) reduced accuracy to 95.46%, and grayscale conversion severely degraded performance to 70.06%, confirming the critical role of RGB color information in mask classification. Third, advanced loss function strategies including Focal Loss, Label Smoothing, and Class Weighted Loss were assessed, with Label Smoothing achieving the closest result to the baseline at 96.07%. Finally, three gradient descent optimization strategies were compared under two fine-tuning configurations, demonstrating that Mini-Batch Gradient Descent provided the most stable convergence and the best generalization performance, particularly under partial fine-tuning of the last five layers.

The findings confirm that the frozen ResNet50 baseline represents a highly efficient and robust framework for face mask classification, and that careful selection of training hyperparameters, regularization techniques, and optimization strategies plays a critical role in achieving reliable real-world performance.

Keywords — Deep Learning, Face Mask Classification, ResNet50, Transfer Learning, Optimization.

I. INTRODUCTION

Face mask classification is becoming more important, particularly in safety and health monitoring systems. In situations where safety regulations must be followed, the ability to automatically classify whether someone is wearing a mask is critical. Traditional methods rely on manual monitoring or rule-based image processing, which are often inefficient. Recent advances in deep learning have enabled models to learn complex facial representations, leading to more accurate and reliable classification. However, face mask classification remains challenging due to variations in lighting, face orientation, occlusion, and mask appearance. Conventional methods struggle to effectively handle such variability. This project uses deep learning techniques to automatically classify masked and unmasked faces. By leveraging transfer learning and convolutional neural networks, the proposed system seeks to develop a robust and accurate face mask classification model. Deep learning is particularly suitable for this task due to its ability to extract high-level visual features while also handling complex real-world variations in images. The objectives of this project are to: (1) implement a deep learning model to classify whether a person is wearing a mask or not, (2) improve the model by training it using images of people wearing masks and people not wearing masks, (3) evaluate the model performance by analyzing the confusion matrix, training and validation accuracy/loss curves, and F1-score, and (4) analyze and interpret the results to assess the model's effectiveness in classifying face masks.

II. METHODOLOGY

2.1 Dataset Description

The dataset utilized in this study is the Face Mask 12K Images Dataset, sourced from Kaggle [1]. It consists of a total of 11,792 high-resolution facial images, structured specifically for binary classification. The dataset is explicitly categorized into two balanced classes: WithMask (individuals wearing face masks) and WithoutMask (individuals not wearing face masks). To ensure robust training and evaluation, the dataset is pre-divided into three distinct subsets: training, validation, and testing. The detailed distribution of images across these subsets is presented in Table 1.

Class	Training	Validation	Testing	Total
With Mask	5,000	400	483	5,883
Without Mask	5,000	400	509	5,909
Total	10,000	800	992	11,792

Table 1: *Dataset Distribution*

All images were resized to 224×224 pixels and normalized by rescaling pixel values from the range [0, 255] to [0, 1] to ensure compatibility with the pre-trained model input requirements. No data augmentation was applied during the baseline phase in order to establish a clean reference benchmark. Due to computational constraints, random sampling was applied to the training set to evaluate the effect of dataset size on model performance, while the full validation and test sets were retained unchanged to ensure fair and consistent evaluation across all experiments

2.2 Pre-trained Model: DeepWeeds ResNet50

The pre-trained weights adopted in this study were sourced from the DeepWeeds project [2], a publicly available deep learning benchmark for automated weed species classification. The DeepWeeds dataset comprises 17,509 images spanning 8 weed species native to pastoral grasslands in Queensland, Australia, along with a negative class, resulting in a total of 9 classification categories. The ResNet50 model trained on this dataset achieved an average classification accuracy of 95.7%, demonstrating robust visual feature extraction capabilities across complex natural environments.

In this work, the DeepWeeds pre-trained weights serve as the initialization point for transfer learning. The final classification layer was replaced to adapt the model to the binary face mask detection task, while all preceding layers were retained to leverage the rich feature representations learned during DeepWeeds training.

2.3 Transfer Learning Architecture

The baseline model adopts the ResNet50 architecture [3], a 50-layer deep residual network originally proposed by He et al. (2016). The model weights were initialized from the DeepWeeds pre-trained model [2], in which ResNet50 was trained on 17,509 weed species images across 9 classes, achieving 95.7% classification accuracy. These pre-trained weights serve as the starting point for transfer learning in this study.

Transfer learning was implemented by freezing all layers of the pre-trained model to preserve the learned feature representations, and replacing the original output layer with a new fully connected Dense layer consisting of 2 neurons with a softmax activation function, corresponding to the two target classes: WithMask and WithoutMask.Only this final classification head was trained on the face mask dataset, resulting in a highly parameter-efficient training procedure, as only the final Dense layer parameters were updated during training while the remaining layers were kept frozen.
The rationale for freezing all layers is grounded in the transferability of low- and mid-level visual features including edge detection, texture recognition, and shape representation which are domain-agnostic and applicable across diverse visual classification tasks. Although the source domain (weed classification) differs substantially from the target domain (face mask detection), both tasks share the requirement of extracting fine-grained discriminative features from complex real-world images, making the transfer of learned representations both justified and effective.

2.4 Training Configuration

The Adam optimizer was selected for model compilation due to its widespread adoption in deep learning classification tasks and its ability to adaptively adjust learning rates for each parameter, yielding stable and efficient convergence in transfer learning scenarios. Categorical cross-entropy was used as the loss function, and classification accuracy served as the primary evaluation metric. Training was conducted with a fixed batch size of 32. The complete hyperparameter configuration for the best-performing baseline experiment is presented in Table 2.

Hyperparameter	**Value**
Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Maximum Epochs	50
Early Stopping Patience	5 epochs
Sample Size (per class)	2,000

Table 2: *Hyperparameter configuration for the best baseline model*

Two training callbacks were employed to ensure optimal model convergence and prevent overfitting. **Early Stopping** monitored the validation loss at each epoch and automatically halted training when no improvement was observed for 5 consecutive epochs, with the best model weights subsequently restored. **ModelCheckpoint** continuously saved the model weights corresponding to the highest validation accuracy achieved throughout training, ensuring that the optimal configuration was preserved regardless of subsequent performance fluctuations.

2.5 Experimental Design

A systematic experimental study was conducted to evaluate the independent effects of three key hyperparameters on classification performance: training sample size, number of training epochs, and learning rate. A total of 12 experiments were performed, with all other hyperparameters held constant to isolate the contribution of each factor. The complete experimental matrix is presented in Table 3.

Exp	Sample Size	Epochs	LR	Test Accuracy
1	100	20	0.001	94.15%
2	500	20	0.001	96.98%
3	1,000	20	0.001	97.38%
4	2,000	20	0.001	98.29%
5	1,000	50	0.001	98.29%
6	2,000	50	0.001	98.89%
7	100	20	0.0001	90.12%
8	1,000	20	0.0001	95.97%
9	2,000	20	0.0001	96.47%
10	500	20	0.01	97.28%
11	1,000	20	0.01	97.48%
12	2,000	50	0.01	98.49%

Table 3: *Experimental configurations and corresponding test accuracy results.*

Experiments 1 through 6 were conducted with a fixed learning rate of 0.001 to evaluate the effect of sample size and number of epochs. Experiments 7 through 9 were conducted with a reduced learning rate of 0.0001, while Experiments 10 through 12 used an increased learning rate of 0.01, to examine the sensitivity of the model to learning rate variations.

The complete source code, experimental configurations, dataset preparation scripts, and trained model weights developed in this study are publicly available at [4].

2.6 Improvement Experiments

A. Model and Input-Based Improvements

To mitigate empirical memorization and enhance the generalization capability of the baseline face mask classification model, a structured regularization framework was introduced while preserving the foundational feature representations stored in the pretrained parameters. The architecture's training behavior was modified by freezing all convolutional and representation layers of the baseline model to prevent feature degradation during backpropagation:

$$layer.trainable = False, \qquad \forall \backslash layer \in M_{\{baseline\}}$$

Let $x \in \mathbb{R}^{\{d\}}$ represent the latent feature vector extracted from the penultimate layer of the frozen baseline architecture $(baseline_{model}.layers[-2].output\,)$ This vector is sequentially transformed through the following regularized pipeline:

1. Batch Normalization: Applied to standardize feature distributions by maintaining approximately zero mean and unit variance across mini-batches, thereby reducing internal covariate shift and stabilizing training.

$$\hat{x} = Batch\,Normalization(x)$$

2. Dropout Regularization: Enforces stochastic regularization by randomly deactivating a subset of neurons (p = 0.2) during training. This prevents excessive reliance on individual activations and reduces neuron co-adaptation.

$$\tilde{x} = Dropout(\hat{x}, p = 0.2)$$

3. Dense Classification Layer: A newly initialized layer ($regularized\_fc\_mask$) with a Softmax activation function maps the regularized space into binary class probabilities. This layer incorporates L_2 weight decay ($\lambda = 10^{-4}$) to penalize large weight magnitudes, encouraging simpler feature representations and limiting overfitting.

$$y = softmax(W\tilde{x} + b)$$

Where W and b represent the trainable weight matrix and bias vector of the newly appended classification layer, respectively. Together, these complementary mechanisms promote the learning of more generalized and robust decision boundaries.

Model Compilation and Hyperparameters

The regularized architecture was optimized using the following experimental configurations:

- Optimizer & Learning Rate: Adam optimizer with a controlled learning rate of $\alpha = 10^{-4}$ to guarantee stable gradient updates across the newly added trainable components.
- Loss Function: Categorical Cross-Entropy loss.
- Training Management: Executed using the validation generator dataset for a maximum of 50 epochs with a batch size of 32.
- Early Stopping & Convergence: To prevent redundant computational iterations and guard against late-stage overfitting, an Early Stopping mechanism was enforced with a patience threshold of 5 epochs, explicitly monitoring the validation loss behavior. During execution, the training successfully converged and triggered early stopping at epoch 33, restoring the optimal parameter weights from epoch 28. The finalized model achieved a robust Test Accuracy of 94.35% and a Test Loss of 0.1460 on the evaluation dataset.

I chose Image Resolution and Color Space Modifications because these factors directly affect the quality of visual information that the model benefits from during training. I wanted to understand how classification accuracy would be affected if I changed the image size or removed the color information.

1. Low Resolution

This method works by resizing the image to a smaller size before feeding it into the model, which reduces the number of pixels and therefore reduces the image details that the model benefits from and learns from. I randomly chose 108 and made sure it was lower than 224 to produce a clear difference in the results. My goal was to see what would happen to the model's performance if I reduced the image size.

2. High Resolution

This method works by enlarging the image before feeding it into the model, which increases the number of pixels and gives the model more visual information that could help it determine whether a person is wearing a mask or not in a better and more efficient way. I randomly chose 300 to provide more details in the image. My goal was the opposite of the first method, to see what would happen if I increased the image size above 224 and whether it could improve the performance or not.

3. Grayscale

This method works by converting the image from three RGB channels to a single grayscale channel, which removes all color information from the image. The image size was kept the same as the original at 224 to isolate the effect of removing color information without introducing any other changes. My goal was to see what would happen if I removed the colors and relied only on grayscale, and whether the model would still be able to distinguish between the two classes or not.

B. Training Strategy Optimization

Loss Function Modifications
The loss function is one of the most important components in deep learning models because it determines how prediction errors are measured and how the model updates its parameters during training. In this experiment, different loss function strategies were investigated to analyze their impact on classification performance and model behavior.

Type 1 — Focal Loss
Focal Loss was selected to address the issue of easy samples dominating the learning process. Instead of treating all training examples equally, this loss function places greater emphasis on samples that are difficult to classify while reducing the influence of correctly classified examples. The parameters were set to gamma = 2.0 and alpha = 0.25. The objective was to determine whether directing more attention toward challenging samples could enhance the model's overall classification performance.

Type 2 — Label Smoothing
Label Smoothing was applied as a regularization technique to reduce prediction overconfidence. Rather than using strict target labels of 0 and 1, a smoothing factor of 0.1 was introduced to create softer target distributions. This encourages the model to make more balanced predictions and may improve its ability to generalize to unseen data. The experiment aimed to evaluate the effect of reducing model confidence on classification accuracy and robustness.

Type 3 — Class Weighted Loss
Class Weighted Loss was used to compensate for potential class imbalance within the dataset. Higher penalty weights were assigned to underrepresented classes, making classification errors on those classes more significant during training. The class weights were calculated automatically based on the dataset distribution. The purpose of this experiment was to investigate whether weighting minority classes could improve fairness and overall classification performance across all categories.

This experiment was conducted to compare the performance of different **Gradient Descent strategies** on the face mask classification task and determine which optimization approach performs best for this specific dataset and classification problem.

Three Gradient Descent strategies were evaluated: **Stochastic Gradient Descent (SGD)**, **Mini-Batch Gradient Descent**, **and Full-Batch Gradient Descent**. The comparison was performed using transfer learning and fine-tuning on a pre-trained **ResNet50 model**.

In **Stochastic Gradient Descent** the model updates its weights using one training sample at a time (**batch size = 1**). This method performs frequent updates during training, which may increase learning randomness but can sometimes improve generalization performance.

In **Mini-Batch Gradient Descent** the model updates its weight using a small subset of the dataset in each iteration. In this experiment a **batch size of 128** was used. This strategy represents a balance between training stability and computational efficiency, making it one of the most commonly used optimization approaches in deep learning.

In **Full-Batch Gradient Descent** the entire training dataset is processed before updating the model weights. In this experiment a **batch size of 800** was used to represent full-batch training. This method provides stable gradient updates but may reduce flexibility during learning and increase training time.

To ensure a fair comparison the **same dataset**, **optimizer**, **learning rate**, and **number of epochs** were used across all experiments. The Adam optimizer with a **learning rate of 0.001** was retained from the baseline configuration. **Early stopping** was also applied to reduce overfitting and stop training when no additional improvement was observed.

The experiments were evaluated under **two fine-tuning settings**. In the **first setting only the final classification layer remained trainable while all previous layers stayed frozen** following the baseline configuration. In the **second setting, the**

last five layers were unfrozen to allow partial fine-tuning and examine whether increasing the number of trainable layers improves adaptation to the face mask classification task.

The main purpose of this experiment was to compare the behavior and performance of different Gradient Descent strategies on this specific task and identify which optimization method achieves the best balance between accuracy, loss reduction, and training stability for face mask classification.

2.7 Evaluation Metrics

Several evaluation metrics were used to evaluate the performance of the proposed face mask classification model and analyze its prediction capability on the testing dataset.

Accuracy was used to measure the percentage of correctly classified images from the total testing dataset, providing an overall indication of model performance.

Loss was used to evaluate the prediction error during training and testing. Lower loss values indicate better convergence and improved learning behavior.

Precision was used to measure the reliability of positive predictions generated by the model and evaluate how accurately the target class was classified.

Recall was used to measure the model's ability to correctly identify images belonging to the target class within the testing dataset.

F1-Score was used to provide a balanced evaluation between Precision and Recall, especially in classification tasks where both metrics are important for performance assessment.

Confusion Matrix was used to visually represent the classification results by illustrating the number of correct and incorrect predictions for each class, helping to analyze model behavior and classification performance in more detail.

III. RESULTS

3.1 Baseline Results

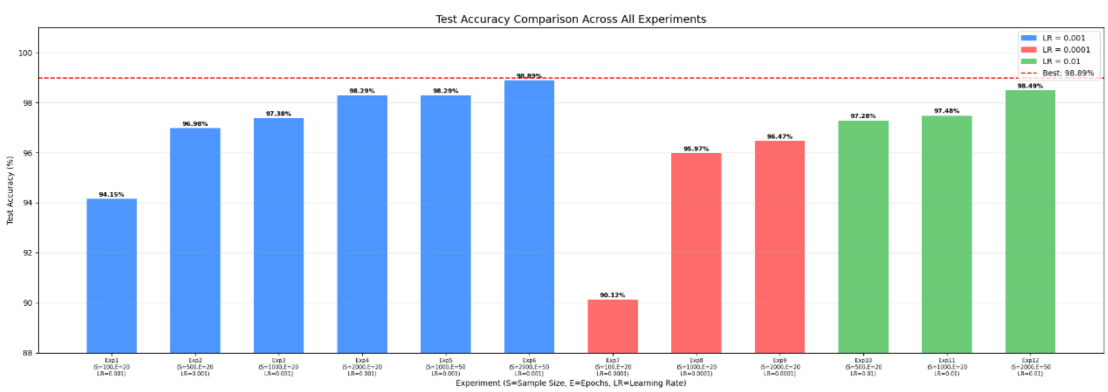

Figure 1: *Test Accuracy Comparison Across All Experiments.*

```
==================================================
Final Results — Best Model
==================================================
Sample Size:     2000
Epochs:          50
Test Accuracy:   98.89%
Test Loss:       0.0312
AUC:             0.9993
F1-Score (W):    0.9889
Precision (W):   0.9889
Recall (W):      0.9889
==================================================
```

Figure 2: *Results of the Best Baseline Model (Exp 6)*

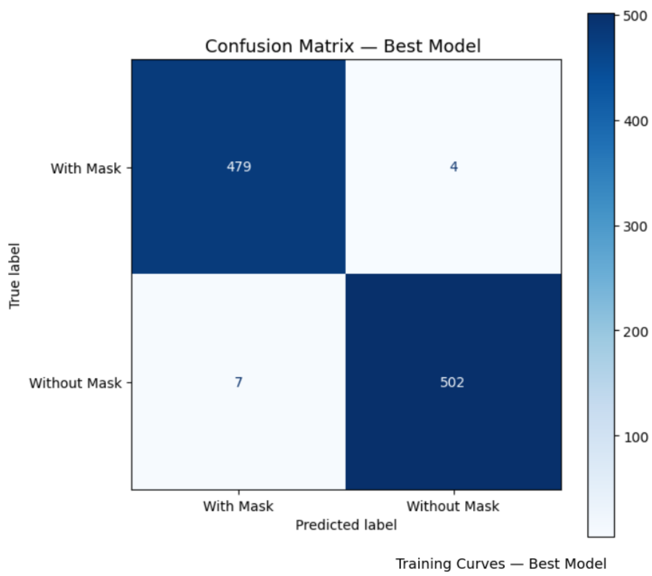

Figure 3: *Confusion Matrix of the Best Baseline Model (Exp 6).*

*As shown in Figure 3, the model correctly classified **479** out of 483 WithMask images and **502** out of 509 WithoutMask images.*

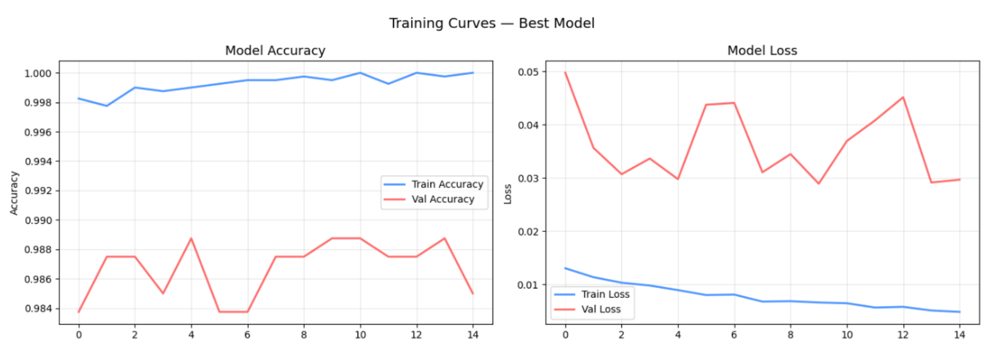

Figure 4: *Training and Validation Accuracy and Loss Curves for the Best Baseline Model (Exp 6).*

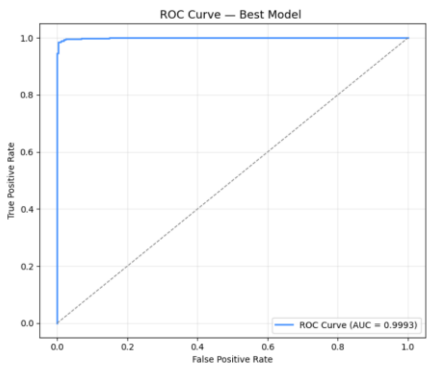

Figure 5*: ROC Curve for the Best Baseline Model (AUC = 0.9993).*

Figure 6*: Per-class Precision, Recall, and F1-Score for the Best Baseline Model (Exp 6).*

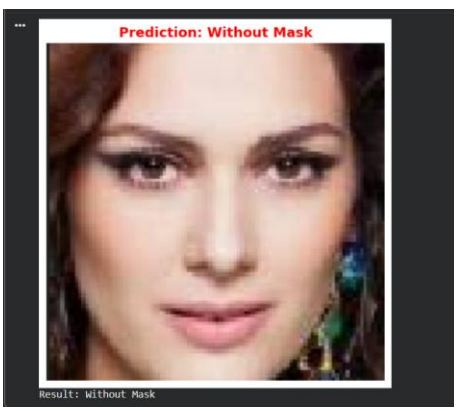

Figure 7: *Baseline model inference result showing a correct "*Without *Mask" prediction.*

Figure 8: *Baseline model inference result showing a correct "With Mask" prediction.*

3.2 Improvement Experiment Results

A. Model and Input-Based Improvements

Figure 9: *Performance Evaluation of the Enhanced Face Mask Classification Model After Applying Regularization-Based Improvements.*

```
========== Evaluation Results ==========
Test Accuracy: 0.9435483813285828
Test Loss: 0.14608831703662872

Classification Report:
              precision    recall  f1-score   support

    WithMask       0.96      0.93      0.94       483
 WithoutMask       0.93      0.96      0.95       509

    accuracy                           0.94       992
   macro avg       0.94      0.94      0.94       992
weighted avg       0.94      0.94      0.94       992
```

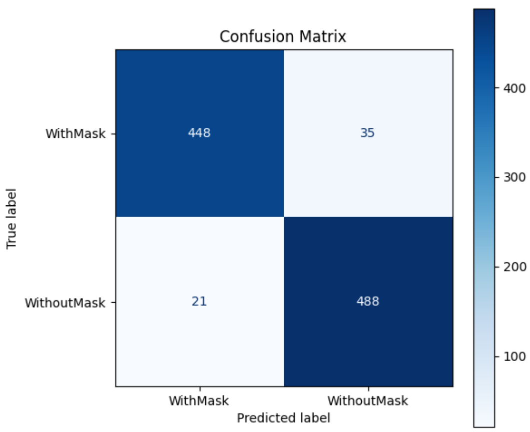

Figure 10: *Confusion Matrix of the Model After Applying Batch Normalization, Dropout, and L2 Regularization.*

The regularized model was subjected to empirical evaluation using the independent test split consisting of 992 images to assess its structural robustness. The enhanced network yielded a stable overall test accuracy of 94.35% alongside a corresponding categorical cross-entropy loss of 0.1460.

The exhaustive breakdown of the model's classification capabilities across both target domains is detailed in the classification report metrics. As shown in **(Figure 9)** For the WithMask class, the network achieved a precision of 0.96 and a recall of 0.93 evaluated against a total support of 483 ground-truth images. Conversely, the WithoutMask class exhibited balanced classification dynamics, reaching a precision of 0.93 and a recall of 0.96 over a support of 509 images.

The explicit decision boundaries are further visualized through the empirical Confusion Matrix **(Figure 10).** Out of the 992 independent samples, the network correctly registered 448 true positive instances of individuals correctly wearing face masks, and 488 true negative instances of individuals without masks. Statistical misclassifications remained critically constrained; only 35 images were subjected to false negative classifications, and 21 instances were erroneously categorized as false positives.

```
========== Evaluation Results ==========
Test Accuracy: 0.9546371102333069
Test Loss: 0.11228223890066147

Classification Report:
              precision    recall  f1-score   support

    WithMask       0.98      0.93      0.95       483
 WithoutMask       0.93      0.98      0.96       509

    accuracy                           0.95       992
   macro avg       0.96      0.95      0.95       992
weighted avg       0.96      0.95      0.95       992
```

Figure 11: *Evaluation results of the Low Resolution experiment (108×108).*

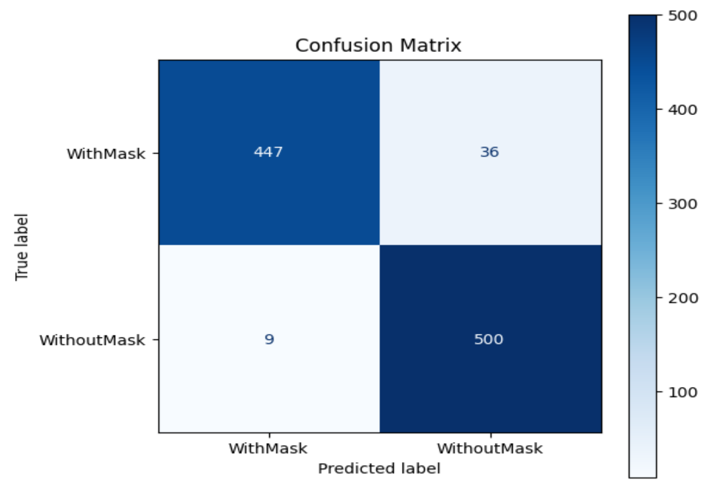

Figure 12: *Confusion matrix of the Low Resolution experiment (108×108).*

```
========== Evaluation Results ==========
Test Accuracy: 0.9879032373428345
Test Loss: 0.03409275785088539

Classification Report:
              precision    recall  f1-score   support

    WithMask       0.99      0.99      0.99       483
 WithoutMask       0.99      0.99      0.99       509

    accuracy                           0.99       992
   macro avg       0.99      0.99      0.99       992
weighted avg       0.99      0.99      0.99       992
```

Figure 13: *Evaluation results of the High Resolution experiment (300×300).*

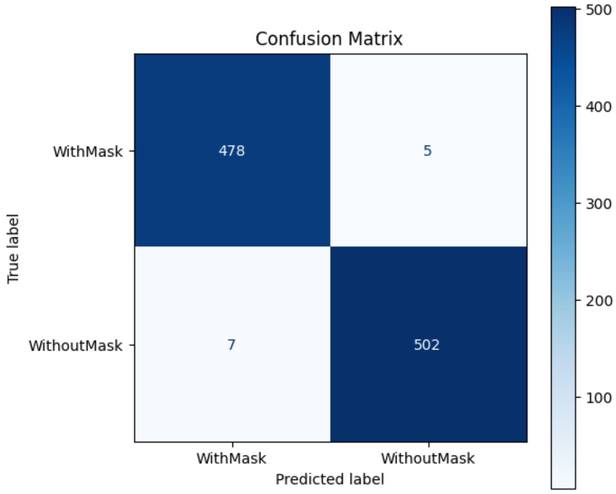

Figure 14: *Confusion matrix of the High Resolution experiment (300×300).*

```
========== Evaluation Results ==========
Test Accuracy: 0.7006048560142517
Test Loss: 0.7070057988166809

Classification Report:
              precision    recall  f1-score   support

    WithMask       0.62      0.99      0.76       483
 WithoutMask       0.99      0.42      0.59       509

    accuracy                           0.70       992
   macro avg       0.80      0.71      0.68       992
weighted avg       0.81      0.70      0.68       992
```

Figure 15: *Evaluation results of the Grayscale experiment (224×224).*

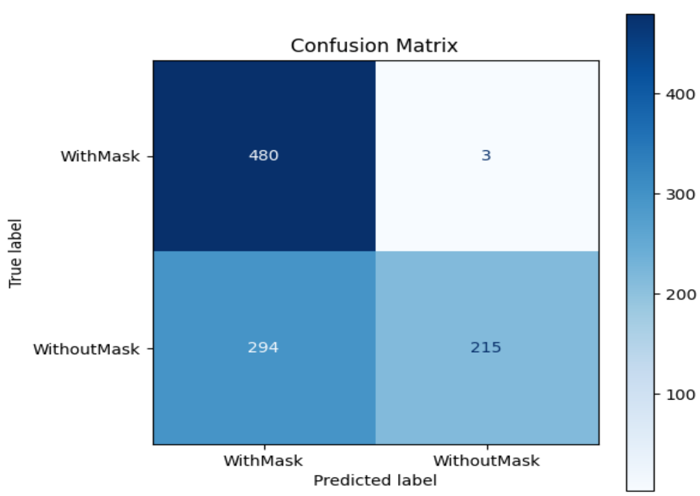

Figure 16: *Confusion matrix of the Grayscale experiment (224×224).*

B. Training Strategy Optimization

```
========== Evaluation Results ==========
Test Accuracy: 0.9536290168762207
Test Loss: 0.007949546910822392

Classification Report:
              precision    recall  f1-score   support

    WithMask       0.95      0.95      0.95       483
 WithoutMask       0.95      0.96      0.95       509

    accuracy                           0.95       992
   macro avg       0.95      0.95      0.95       992
weighted avg       0.95      0.95      0.95       992
```

Figure 17: *Training and validation performance using Focal Loss*

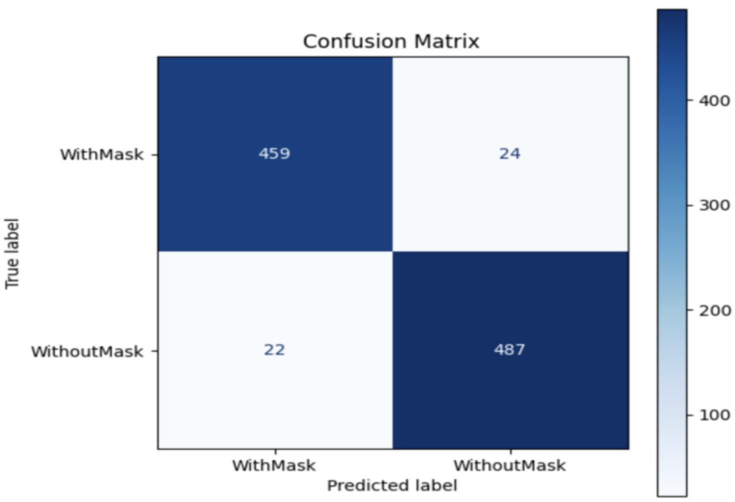

Figure 18: *Confusion matrix for the model trained with Focal Loss*

```
========== Evaluation Results ==========
Test Accuracy: 0.9606854915618896
Test Loss: 0.2856581509113312

Classification Report:
              precision    recall  f1-score   support

    WithMask       0.96      0.95      0.96       483
 WithoutMask       0.96      0.97      0.96       509

    accuracy                           0.96       992
   macro avg       0.96      0.96      0.96       992
weighted avg       0.96      0.96      0.96       992
```

Figure 19: *Training and validation performance using Label Smoothing.*

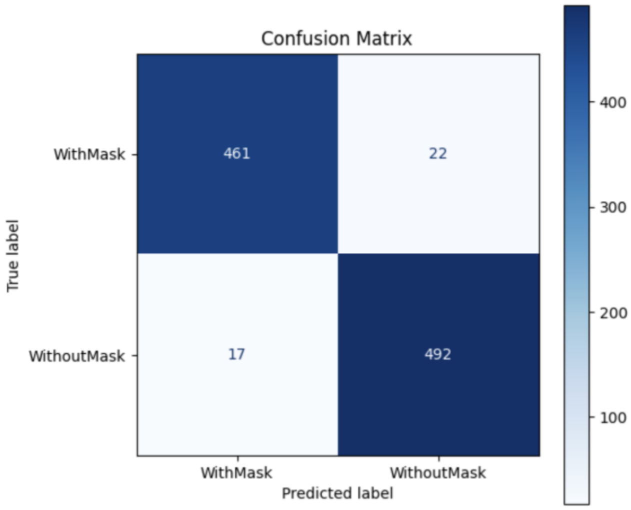

Figure 20: *Confusion matrix for the Label Smoothing model.*

```
========== Evaluation Results ==========
Test Accuracy: 0.9546371102333069
Test Loss: 0.12674370408058167

Classification Report:
              precision    recall  f1-score   support

    WithMask       0.96      0.94      0.95       483
 WithoutMask       0.95      0.96      0.96       509

    accuracy                           0.95       992
   macro avg       0.95      0.95      0.95       992
weighted avg       0.95      0.95      0.95       992
```

Figure 21: *Training and validation performance using Class Weighted Loss.*

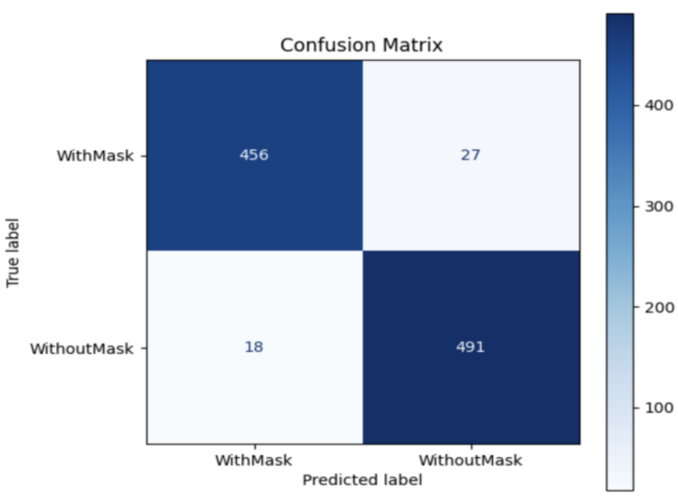

Figure 22: *Confusion matrix for the Class Weighted Loss model.*

The experimental results showed that the **Gradient Descent strategies** produced different behaviors and performance levels across the face mask classification task. The comparison was conducted using two fine-tuning configurations: training only the final classification layer and training the last five layers of the ResNet50 model.

When only the **final classification layer was trainable**, all three strategies achieved very high classification performance with test accuracy close to 99%, as illustrated in **Figure 29. Full-Batch Gradient Descent** achieved the highest **accuracy of 98.79%** with the **lowest test loss of 0.0359**. **Stochastic Gradient Descent** achieved an **accuracy of 98.69%** with a **loss of 0.0538**, while **Mini-Batch Gradient Descent** achieved **98.59% accuracy** with a **loss of 0.0432**. These results indicate that all strategies performed effectively under the baseline fine-tuning configuration.

When the **last five layers were unfrozen for partial fine-tuning**, the performance differences became more noticeable as shown in **Figure 29 and 30**. **Mini-Batch Gradient Descent** maintained stable performance and achieved the best overall results among the partial fine-tuning experiments, with **test accuracy of 98.59%** and a **low-test loss of 0.0449**. In comparison **Stochastic Gradient Descent** showed a reduction in performance, achieving **94.66% accuracy** with a significantly **higher loss value of 0.4987**. Although **Full-Batch Gradient Descent** achieved **lower classification accuracy**, **it still maintained a reasonable test loss of 0.1895** which was lower than the loss obtained by the Stochastic Gradient Descent experiment with five trainable layers.

The confusion matrices further demonstrated the classification behavior of each strategy. The experiments using only the final classification layer produced highly balanced predictions with very few misclassified samples. However, after unfreezing the last five layers, both Stochastic Gradient Descent and Full-Batch Gradient Descent showed an increase in classification errors and class imbalance, especially for the WithMask category, as illustrated in **Figures 31 and 33**.

```
========== Evaluation Results ==========
Test Accuracy: 0.9868951439857483
Test Loss: 0.0538468137383461

Classification Report:
              precision    recall  f1-score   support

    WithMask       0.98      0.99      0.99       483
 WithoutMask       0.99      0.98      0.99       509

    accuracy                           0.99       992
   macro avg       0.99      0.99      0.99       992
weighted avg       0.99      0.99      0.99       992
```

Figure 23: *Evaluation results of the Stochastic Gradient Descent strategy using the baseline training configuration*

```
========== Evaluation Results ==========
Test Accuracy: 0.9858871102333069
Test Loss: 0.04315580800175667

Classification Report:
              precision    recall  f1-score   support

    WithMask       0.98      0.99      0.99       483
 WithoutMask       0.99      0.98      0.99       509

    accuracy                           0.99       992
   macro avg       0.99      0.99      0.99       992
weighted avg       0.99      0.99      0.99       992
```

Figure 24: *Evaluation results of the Mini-Batch Gradient Descent strategy using the baseline training configuration*

```
========== Evaluation Results ==========
Test Accuracy: 0.9879032373428345
Test Loss: 0.035935357213020325

Classification Report:
              precision    recall  f1-score   support

    WithMask       0.98      0.99      0.99       483
 WithoutMask       0.99      0.98      0.99       509

    accuracy                           0.99       992
   macro avg       0.99      0.99      0.99       992
weighted avg       0.99      0.99      0.99       992
```

Figure 25: *Evaluation results of the Full-Batch Gradient Descent strategy using the baseline training configuration*

```
========== Evaluation Results ==========
Test Accuracy: 0.9465726017951965
Test Loss: 0.49865391850471497

Classification Report:
              precision    recall  f1-score   support

    WithMask       0.92      0.97      0.95       483
 WithoutMask       0.97      0.92      0.95       509

    accuracy                           0.95       992
   macro avg       0.95      0.95      0.95       992
weighted avg       0.95      0.95      0.95       992
```

Figure 26: *Evaluation results of the Stochastic Gradient Descent strategy after unfreezing the last five trainable layers*

```
========== Evaluation Results ==========
Test Accuracy: 0.9858871102333069
Test Loss: 0.0449117049574852

Classification Report:
              precision    recall  f1-score   support

    WithMask       0.98      0.99      0.99       483
 WithoutMask       0.99      0.98      0.99       509

    accuracy                           0.99       992
   macro avg       0.99      0.99      0.99       992
weighted avg       0.99      0.99      0.99       992
```

Figure 27: *Evaluation results of the Mini-Batch Gradient Descent strategy after unfreezing the last five trainable layers*

```
========== Evaluation Results ==========
Test Accuracy: 0.9223790168762207
Test Loss: 0.18952502310276031

Classification Report:
              precision    recall  f1-score   support

    WithMask       1.00      0.84      0.91       483
 WithoutMask       0.87      1.00      0.93       509

    accuracy                           0.92       992
   macro avg       0.93      0.92      0.92       992
weighted avg       0.93      0.92      0.92       992
```

Figure 28: *Evaluation results of the Full-Batch Gradient Descent strategy after unfreezing the last five trainable layers*

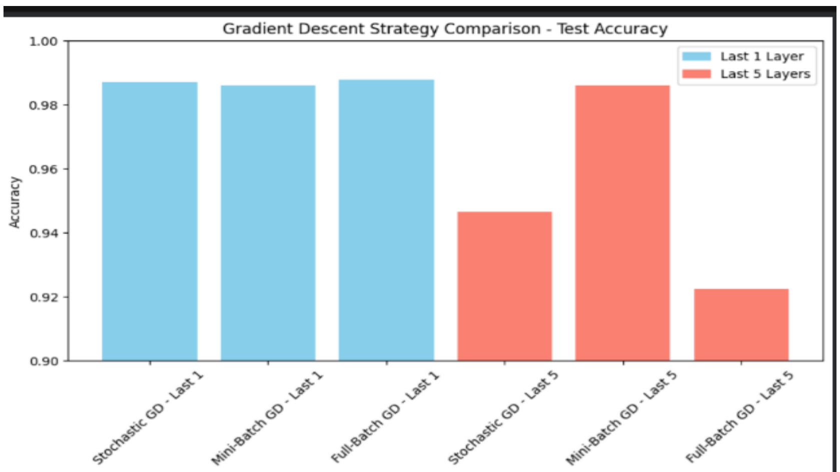

Figure 29: *Comparison of test accuracy across different Gradient Descent strategies using the baseline configuration and partial fine-tuning setup*

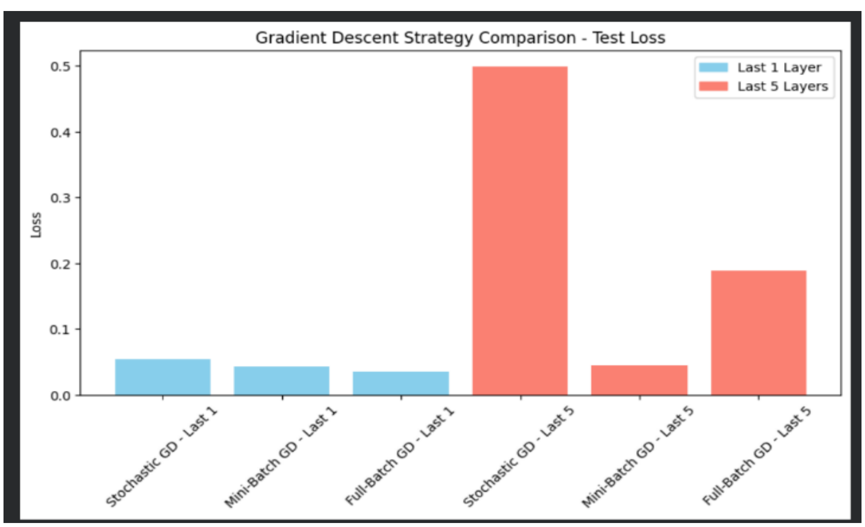

Figure 30: *Comparison of test loss across different Gradient Descent strategies*

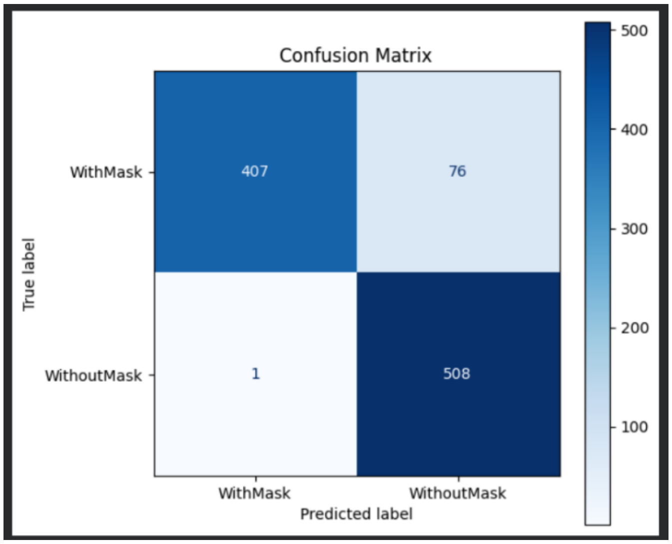

Figure 31: *Confusion matrix results of the Full-Batch Gradient Descent strategy after unfreezing the last five*

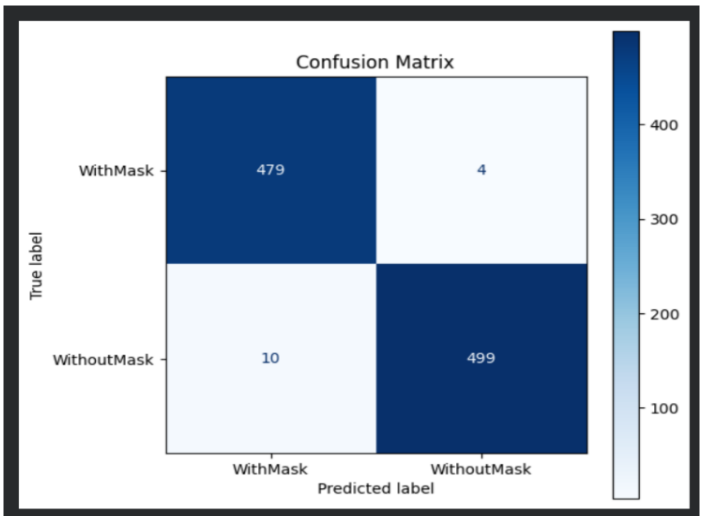

Figure 32: *Confusion matrix results of the Mini-Batch Gradient Descent strategy after unfreezing the last five*

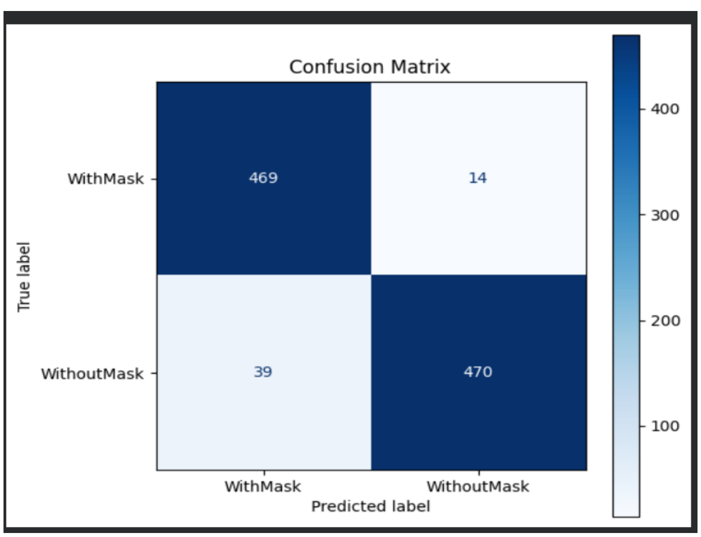

Figure 33: *Confusion matrix results of the Stochastic Gradient Descent strategy after unfreezing the last five trainable layers*

IV. ANALYSIS AND DISCUSSION

4.1 Baseline Performance Analysis

The baseline model demonstrated strong and consistent classification performance across the face mask detection task, achieving a test accuracy of 98.89%, a test loss of 0.0312, an AUC of 0.9993, and a weighted F1-Score, Precision, and Recall of 0.9889, as reported in Fig. 2. These results confirm that transfer learning from a domain-different source can effectively generalize to a new classification task, even when the source domain (weed classification) differs substantially from the target domain (face mask detection).

Effect of Sample Size: As illustrated in Fig. 1 and Table 3, a consistent positive relationship was observed between training sample size and classification accuracy. Under a fixed learning rate of 0.001 and 20 training epochs, accuracy increased progressively from 94.15% in Experiment 1 (100 samples) to 96.98% in Experiment 2 (500 samples), 97.38% in Experiment 3 (1,000 samples), and 98.29% in Experiment 4 (2,000 samples). This progressive improvement confirms that larger training sets provide richer and more diverse feature representations, enabling the classification head to learn a more robust and generalizable decision boundary.

Effect of Training Epochs: Increasing the number of training epochs from 20 to 50 with a fixed sample size of 2,000 images per class (Exp 4 vs. Exp 6) yielded a further improvement from 98.29% to 98.89%, demonstrating that extended training with sufficient data enables better convergence and more refined weight optimization. The training and validation curves presented in Fig. 4 confirm smooth and stable convergence throughout training, with early stopping triggered at epoch 28 and the best model weights successfully restored. A notable observation is that Experiment 5 (1,000 samples, 50 epochs) achieved the same accuracy as Experiment 4 (2,000 samples, 20 epochs), both reaching 98.29%, revealing a meaningful trade-off between dataset size and training duration, where additional epochs can partially compensate for a smaller training set.

Effect of Learning Rate: As illustrated in Fig. 1, the learning rate had a significant impact on classification accuracy. A learning rate of 0.0001 consistently produced the lowest accuracy values across all configurations, reaching a minimum of 90.12% in Experiment 7, which is attributed to insufficient weight updates that prevented the model from converging adequately within the allocated training epochs. In contrast, LR = 0.01 achieved competitive accuracy in several experiments, reaching up to 98.49% in Experiment 12, however LR = 0.001 achieved higher overall accuracy of 98.89% in Experiment 6 under the same sample size and epoch configuration, and was therefore identified as the optimal learning rate for the baseline model.

Overall Performance: The confusion matrix presented in Fig. 3 confirmed highly balanced classification behavior across both target classes. Out of 992 test images, the model correctly classified 479 out of 483 WithMask images and 502 out of 509 WithoutMask images, resulting in only 4 false negatives and 7 false positives. The ROC curve in Fig. 5 yielded an AUC of 0.9993, indicating near-perfect discriminative ability across all classification thresholds. The per-class metrics presented in Fig. 6 further confirmed balanced performance, with precision, recall, and F1-score of 0.989 achieved across both classes, confirming that the model did not exhibit any significant bias toward either class. Finally, the qualitative inference results presented in Fig. 7 and Fig. 8 demonstrate the model's ability to correctly classify real-world facial images under varying conditions, further validating its practical applicability.

4.2 Improvement Analysis

A. Model and Input-Based Improvements Analysis

A comparative analytical review between the baseline performance and the newly introduced framework reveals a marginal nominal decline in empirical testing accuracy, shifting from the baseline's 99.09% to the enhanced model's 94.35%. In deep learning paradigms, an empirical accuracy approaching near-perfection (> 99%) on localized datasets indicates an under-regularized architecture prone to data memorization (overfitting). The baseline model optimized its decision boundaries without structural constraints, thereby absorbing high-frequency patterns specific to the training distribution.

Although the enhanced model achieved a lower test accuracy than the baseline, the observed 4.74% reduction should not necessarily be interpreted as a deterioration in model quality. Instead, it reflects the intended effect of regularization techniques that constrain model complexity and reduce dependence on training-specific patterns. Near-perfect accuracy may indicate partial memorization of dataset characteristics, whereas the regularized model was encouraged to learn more transferable feature representations.

The systematic integration of Batch Normalization, Dropout ($p = 0.2$), and L_2 weight decay introduced regularization constraints intended to reduce the model complexity and mitigate the risk of overfitting. By penalizing large weight coefficients via L_2 regularization and reducing co-adaptation through Dropout, the network was encouraged to learn more robust feature representations.

The resulting 94.35% accuracy, together with the balanced precision and recall values across both classes, demonstrates that the regularized model maintained strong classification performance despite the reduction in overall accuracy. These findings suggest that the introduced regularization techniques preserved robust predictive behavior while encouraging a more conservative learning framework. Consequently, the improved architecture offers a more realistic estimate of performance under unseen conditions and is expected to show greater robustness when deployed in practical environments where data distributions may differ from those observed during training.

First, I chose the category Image Resolution & Color Space Modifications with its three types:

1.Low Resolution

2.High Resolution

3.Grayscale Transformation

To improve the baseline model.

The comparison is based on the baseline results:

- Test Accuracy: 98.89%
- Test Loss: 0.0312
- F1-Score (W): 0.9889
- Precision (W): 0.9889
- Recall (W): 0.9889

I started with the first method, Low Resolution. I asked myself: what would happen to the metrics if I reduced the image size below 224? I chose to set the image size to 108 and kept the same number of epochs (50). I trained and tested the method on the validation and test images. After running, I got the following results:

- Test Accuracy: 95.46%
- Test Loss: 0.1123
- F1-Score (W): 0.95
- Precision (W): 0.96
- Recall (W): 0.95

Regarding the accuracy (95.46% vs 98.89%), the result was relatively good but still lower than the baseline. The reason is that the Low Resolution method gave a lower score because I reduced the image size to 108, which led to losing facial details and mask details in the image.

As for the Test Loss (0.1123 vs 0.0312), the baseline model achieved a lower value, which means the baseline was more confident in its decisions and made fewer errors on the test data compared to the Low Resolution model.

The F1-Score (0.95 vs 0.9889) of the improved Low Resolution model was lower than the baseline because both Recall (0.95 vs 0.9889) and Precision (0.96 vs 0.9889) decreased. The reason for the Precision drop is that the model was accurate in classifying WithMask but made errors on some images due to weak details. The reason for the Recall drop is that the model missed some correct images because the small image size did not provide enough information.

I noticed that the reason all metrics dropped in the Low Resolution method is because reducing the image size made it harder for the model to produce better results than the baseline.

I then moved to the second method High Resolution. I asked myself: what would happen if I increased the image size above 224? I chose to set the image size to 300 and kept the same number of epochs (50). I trained and tested the method on the validation and test images. After running, I got the following results:

- Test Accuracy: 98.79%
- Test Loss: 0.0340
- F1-Score (W): 0.99
- Precision (W): 0.99
- Recall (W): 0.99

Starting with the accuracy (98.79% vs 98.89%), I noticed it was very close to the baseline. This is evidence that the High Resolution method was able to get very close to the baseline accuracy. The reason is that increasing the image size led to images containing more details, which helped the model distinguish between a person wearing a mask and a person not wearing one.

As for the Test Loss (0.0340 vs 0.0312), it was also very close to the baseline value, which means the model was almost as confident in its decisions as the baseline model, with only a slight difference in errors on the test data.

The F1-Score (0.99 vs 0.9889) matched the baseline, and both Precision (0.99 vs 0.9889) and Recall (0.99 vs 0.9889) also matched. The reason is that increasing the image size to 300 provided the model with richer visual information, which helped it correctly identify both classes with high confidence, resulting in strong Precision and Recall scores similar to the baseline.

In summary, the High Resolution method achieved results very close to the baseline. Some metrics matched the baseline exactly and others were very close, which confirms that increasing the image size to a reasonable number provides more details in the image.

I then moved to the last method Grayscale. I asked myself: what would happen to the accuracy if I removed the RGB color channels? Of course, I did not change the image size and kept it at 224. I also kept the same number of epochs and trained and tested the method on the validation and test images. After running, the results were:

- Test Accuracy: 70.06%
- Test Loss: 0.7070
- F1-Score (W): 0.68
- Precision (W): 0.81
- Recall (W): 0.70

The accuracy (70.06% vs 98.89%) was very bad. The reason is clear when I removed the colors, I removed important information that the model relies on. This shows that the presence of colors in the image helps more in classifying whether a person is wearing a mask or not.

As for the Test Loss (0.7070 vs 0.0312), this was the worst result across all methods. The huge gap between the Grayscale model and the baseline shows that without color information, the model became very uncertain in its decisions and made significantly more errors on the test data.

The F1-Score (0.68 vs 0.9889) was very bad, even worse than the accuracy, because the Recall (0.70) dropped significantly. The reason is that the model missed many images that it could have identified correctly with colors. As for the Precision (0.81), although it gave a higher value than the Recall, this does not mean it is better. Looking at the Confusion Matrix, we can see that the model was biased toward the WithMask class, meaning it classified most images as WithMask regardless of the actual label, which deflated the WithMask Precision artificially and inflated the WithoutMask Precision artificially.

I noticed that when I removed the colors, the metrics were heavily affected some showed bias and others dropped significantly. This proves the importance of colors in the image.

Finally, all three methods Low Resolution, High Resolution, and Grayscale were not able to improve the baseline. Some performed poorly and some got close to the baseline results, but the baseline remains the best. The reason is:

1.The baseline was trained on 224x224, the optimal size for ResNet50

2.The baseline was trained on 2000 images more than the validation set (800 images)

3.The baseline used full RGB colors no information was removed

B. Training Strategy Optimization Analysis

Loss Function Strategies

1. Comparison with the Baseline Model

The baseline ResNet50 model recorded a test accuracy of **99.09%** and a test loss of **0.0319**, establishing a strong reference point for evaluating the proposed loss function modifications. These results confirm that the original transfer learning setup was highly capable of handling the face mask classification task.

Upon applying the three loss function modifications, all strategies fell short of baseline accuracy. Label Smoothing yielded the closest result to the baseline among the three approaches with an accuracy of **96.07%,** while Class Weighted Loss and Focal Loss followed with **95.46%** and **95.36%,** respectively. The consistent drop in accuracy across all modifications suggests that altering the loss function alone, without unfreezing any of the pre-trained layers, was insufficient to surpass the existing performance level.

These findings indicate that the standard cross-entropy loss function was already well-aligned with the nature of this classification task, and the feature representations inherited from the pre-trained ResNet50 model were robust enough to achieve high accuracy without requiring any loss function adjustments.

---

2. Comparison Between Loss Function Strategies

The three loss function strategies demonstrated distinct behaviors in terms of classification accuracy and loss values throughout the experiments.

Among the three modifications, Label Smoothing delivered the strongest classification performance with a test accuracy of **96.07%.** Despite this, its test loss reached **0.2857**, considerably higher than the baseline. This outcome is consistent with how Label Smoothing works, as replacing hard labels with softer values inherently prevents the loss from converging to very low numbers. Nevertheless, the model was still able to classify samples with high accuracy, which reflects the regularization benefit that Label Smoothing provides by discouraging overconfident predictions.

Focal Loss, on the other hand, recorded the lowest test loss across all conducted experiments at 0.0079, falling even below the baseline loss of **0.0319**. This result demonstrates that directing the model's attention toward difficult misclassified samples effectively reduced the overall training error. However, this reduction in loss did not correspond to an improvement in classification accuracy, as Focal Loss reached only **95.36%.** This can be attributed to the fact that the baseline model already performed at a high level, leaving very few challenging samples for Focal Loss to focus on.

Class Weighted Loss produced a test accuracy of **95.46%** alongside a test loss of **0.1267**. The relatively moderate loss value reflects the additional penalty constraints introduced by the class weights during training. Given that the face mask dataset maintained a reasonably balanced distribution between the two classes, the advantages offered by class weighting were not fully realized, resulting in performance comparable to Focal Loss.

In summary, although Label Smoothing proved to be the most effective among the three loss modifications, none of the tested strategies managed to surpass the baseline model. This outcome highlights that the standard cross-entropy loss function was already well-matched to the characteristics of this dataset, and the robust feature representations extracted by the pre-trained ResNet50 were sufficient for achieving high classification performance without the need for advanced loss function adjustments.

Gradient Descent Strategies

1. Comparison with the Baseline Model

The baseline ResNet50 model achieved strong classification performance with a **test accuracy of 98.89%** and a **low-test loss of 0.0312**. The baseline configuration also demonstrated stable convergence behavior during training, indicating that the original transfer learning setup was already highly effective for the face mask classification task.

When the Gradient Descent strategies were evaluated using only the final trainable layer, all experiments produced results close to the baseline performance. Full-Batch Gradient Descent achieved the closest performance to the baseline with an **accuracy of 98.79%** and **a loss of 0.0359**, while Mini-Batch and Stochastic Gradient Descent also maintained classification **accuracies above 98%.**

These results indicate that when most layers of the pre-trained ResNet50 model remain frozen, the optimization strategy has a limited effect on overall classification performance. In this configuration, the model mainly relies on the robust feature representations already learned during pre-training, which explains why all strategies achieved similar results.

However, after unfreezing the last five layers, the differences between the optimization strategies became more significant. Mini-Batch Gradient Descent maintained performance very close to the baseline model, achieving **98.59% accuracy** with a **low loss value of 0.0449**. This indicates that Mini-Batch optimization was able to fine-tune the additional trainable layers effectively without significantly affecting the model's generalization capability.

In contrast, both Stochastic and Full-Batch Gradient Descent showed noticeable reductions in classification accuracy after increasing the number of trainable layers. This suggests that additional fine-tuning introduced more complex optimization behavior, making the model more sensitive to the selected gradient update strategy.

Overall, the baseline configuration remained one of the strongest performing setups for this task, achieving the highest overall balance between classification accuracy and low-test loss. This indicates that the original transfer learning configuration of the ResNet50 model was already highly optimized for the face mask classification dataset. While some optimization strategies produced competitive results, none of the additional experiments significantly outperformed the baseline model in overall performa.

2. Comparison Between Gradient Descent Strategies

The comparison between the three Gradient Descent strategies revealed clear differences in convergence stability, loss behavior, and generalization performance after partial fine-tuning.

As shown in **Figures 34 and 35**, Mini-Batch Gradient Descent produced the most balanced performance among all experiments. The strategy achieved stable classification accuracy while maintaining lower test loss compared to the other optimization methods. This stability can be attributed to the use of moderate batch updates, which reduces the randomness of weight updates while still allowing the model to adapt gradually during fine-tuning. As a result, Mini-Batch Gradient Descent achieved strong classification performance and stable generalization behavior.

Stochastic Gradient Descent behaved differently after unfreezing the last five layers. Although the training accuracy rapidly approached very high values, the test accuracy decreased and the test loss increased significantly. This behavior indicates that the model became highly sensitive to noisy weight updates caused by processing one sample at a time. As more layers became trainable, these highly frequent updates caused unstable optimization behavior and reduced the model's ability to generalize effectively to unseen testing data. This explains the large increase in loss observed in **Figure 34**.

A similar but less severe trend appeared in the Full-Batch Gradient Descent experiment. The training process remained relatively stable, and the loss values remained relatively stable because the weights were updated using the entire dataset at once. However, despite this stability, the classification accuracy still decreased after unfreezing the last five layers. This suggests that Full-Batch updates reduced the flexibility of the optimization process during fine-tuning. Since the model waited for the entire dataset before performing updates, it adapted more slowly to task-specific features, leading to weaker classification performance despite maintaining moderate loss values.

The confusion matrices shown in **Figures 31, 32 and 33** further support these observations. Mini-Batch Gradient Descent produced highly balanced predictions with very few classification errors, while Stochastic and Full-Batch Gradient Descent showed larger numbers of misclassified samples after increasing the number of trainable layers.

Overall, the analysis demonstrates that Mini-Batch Gradient Descent was the most suitable optimization strategy for this face mask classification task. It achieved the best balance between convergence stability, classification accuracy, loss reduction, and generalization performance, especially when partial fine-tuning was applied to the ResNet50 model.

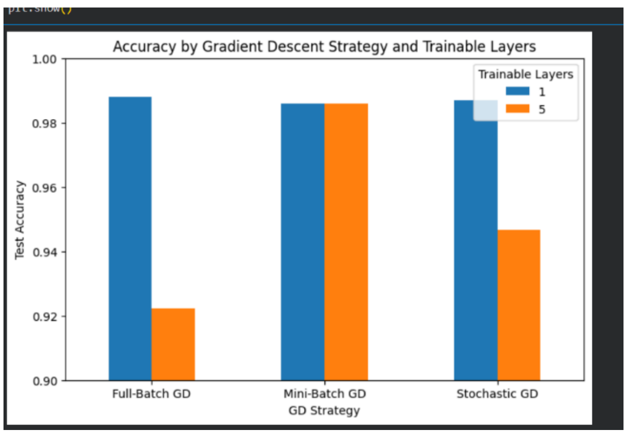

Figure 34: *Comparison of test accuracy across different Gradient Descent strategies using one trainable layer and partial fine-tuning with the last five trainable layers*

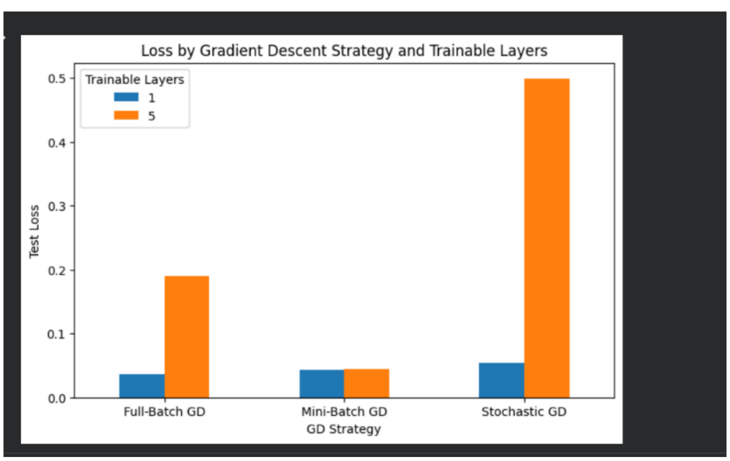

Figure 35: *Comparison of test loss across different Gradient Descent strategies using one trainable layer and partial fine-tuning with the last five trainable layers*

V. CONCLUSIONS

This study demonstrated that transfer learning using a pre-trained ResNet50 model from the DeepWeeds dataset can effectively generalize to face mask classification, achieving a baseline test accuracy of 98.89%, AUC of 0.9993, and weighted F1-Score of 0.9889. Systematic experimentation confirmed that larger training sets, extended epochs, and a learning rate of 0.001 collectively yielded the best classification performance.

Improvement experiments revealed that while regularization techniques reduced accuracy to 94.35%, they promoted better generalization. Image resolution experiments confirmed that the standard 224×224 RGB input is optimal for ResNet50, as grayscale conversion severely degraded performance to 70.06%. Among loss function strategies, Label Smoothing achieved the closest result to the baseline at 96.07%, and Mini-Batch Gradient Descent provided the most stable convergence under partial fine-tuning.

Overall, the frozen ResNet50 baseline proved to be a highly efficient and robust framework for this task. Future work may explore deeper fine-tuning strategies, advanced data augmentation, and real-time deployment on edge devices.

REFERENCES

[1] P. Jangra, "Face Mask 12K Images Dataset," Kaggle, 2020. [Online]. Available: https://www.kaggle.com/datasets/ashishjangra27/face-mask-12k-images-dataset

[2] A. Olsen, D. A. Konovalov, B. Philippa, P. Ridd, J. C. Wood, J. Johns, W. Banks, B. Girgenti, O. Kenny, J. Whinney, B. Calvert, M. Rahimi Azghadi, and R. D. White, "DeepWeeds: A Multiclass Weed Species Image Dataset for Deep Learning," *Scientific Reports*, vol. 9, no. 1, p. 2058, Feb. 2019. [Online]. Available: https://doi.org/10.1038/s41598-018-38343-3. GitHub: https://github.com/AlexOlsen/DeepWeeds

[3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778. [Online]. Available: https://arxiv.org/abs/1512.03385

[4] R. Althagafi, "Face Mask Detection using Deep Learning and Transfer Learning," GitHub, 2025. [Online]. Available: https://github.com/RemaAlthagafi/face-mask-detection-deep-learning